

CRIPTOGRAFÍA Y BLOCKCHAIN

Módulo 3

Criptografía moderna

Fundamentos, clasificación, criptografía simétrica, bits de seguridad, bloque y flujo, modos, padding, AES, ChaCha20, RSA, curvas elípticas, claves de sesión y laboratorio de contraseñas.

Material elaborado por el profesor Sergio Gevatschnaider

Contenido

Objetivos de aprendizaje	4
Convenciones	4
1. Fundamentos	4
Seguridad perfecta y computacional	4
2. Clasificación de criptosistemas modernos	5
Qué significa integridad	5
3. Criptografía simétrica	5
AES	5
ChaCha20	5
Gestión del secreto compartido	5
4. Bits de seguridad	6
Impacto cuántico	6
Tabla orientativa	6
Ejemplo de cálculo	6
5. Cifrado por bloques y por flujo	7
Bloques	7
Flujo	7
6. Modos de operación	7
Nonce, IV y contador	7
Encrypt-then-MAC y AEAD	7
7. Padding	7
Ejemplo PKCS#7	8
8. Algoritmos simétricos	8
AES	8
ChaCha20	8
Vector de control RFC 8439	8
Reutilización de nonce en ChaCha20	9
ChaCha20-Poly1305	9
DES y 3DES/TDEA	9
Selección práctica	9

9. Criptografía asimétrica	9
10. RSA y curvas elípticas	10
RSA	10
ECC	10
Comparación funcional	10
11. Claves de sesión	10
Separación y contexto	11
Secreto hacia adelante	11
12. Laboratorio: vulnerar contraseña	11
Estrategias	11
Salt	11
KDF de contraseña	11
Cadena de ataque	11
Recomendaciones	12
Laboratorio seguro	12
13. Casos de diseño	12
Caso A: archivo local protegido por contraseña	12
Caso B: canal cliente-servidor	12
Caso C: mensaje a una clave pública	12
14. Errores frecuentes	13
15. Actividades y autoevaluación	13
Actividad 1: clasificación	13
Actividad 2: bits de seguridad	13
Actividad 3: nonce repetido	13
Actividad 4: arquitectura híbrida	13
Preguntas de control	13
16. Lista de comprobación curricular	14
Recursos interactivos nuevos	14
Recursos interactivos existentes	14
Bibliografía y estándares	14

La criptografía moderna combina primitivas matemáticas, esquemas, protocolos e ingeniería de seguridad. El objetivo no es únicamente ocultar datos: debe declarar qué propiedad protege, frente a qué adversario, con qué recursos, bajo qué supuestos y durante cuánto tiempo.

Recorrido del módulo: índice · teoría HTML · biblioteca PDF por tema · laboratorio ChaCha20 · mapas mentales · glosario integral · cuestionario integral

Objetivos de aprendizaje

Al terminar el módulo, el estudiante podrá:

1. Formular una afirmación de seguridad indicando propiedad, adversario, recursos y supuestos.
2. Clasificar primitivas, esquemas, protocolos y sistemas sin confundir sus garantías.
3. Comparar cifradores por bloques y de flujo, y explicar el papel de modos, padding, nonces, AAD y tags.
4. Justificar el uso de AEAD y detectar configuraciones frágiles como ECB, CTR sin MAC o reutilización de nonce.
5. Explicar AES y ChaCha20, verificar un vector conocido y reconocer el límite de un cifrador sin autenticación.
6. Distinguir transporte de claves con RSA-OAEP, acuerdo ECDH/X25519 y encapsulación mediante KEM.
7. Construir el razonamiento de una sesión híbrida: autenticación, establecimiento, KDF, AEAD, rotación y destrucción.
8. Analizar un ataque offline a contraseñas y elegir una defensa basada en costo, entropía y operación.

Convenciones

- **M**: mensaje o texto claro.
- **C**: texto cifrado.
- **K**: clave secreta.
- **N**: nonce, valor que no debe repetirse bajo la misma clave cuando la construcción así lo exige.
- **AAD**: datos asociados autenticados pero no cifrados.
- **tag**: etiqueta de autenticación.
- **XOR**: operación exclusiva-or bit a bit.

1. Fundamentos

Una definición moderna de seguridad debe especificar:

1. **Propiedad**: confidencialidad, integridad, autenticidad, no repudio, anonimato u otra.
2. **Adversario**: capacidades, acceso, información auxiliar y oráculos.
3. **Condición de victoria**: distinguir, recuperar, falsificar, alterar o correlacionar.
4. **Recursos**: tiempo, memoria, datos, consultas, energía y paralelización.
5. **Supuesto**: problema difícil, modelo de aleatoriedad, custodia de claves y entorno de ejecución.

El principio de Kerckhoffs establece que el algoritmo puede ser público. La seguridad no debe depender de ocultar el diseño.

Seguridad perfecta y computacional

- **Secreto perfecto**: observar el criptograma no modifica la distribución del mensaje. El one-time pad lo alcanza solo con clave uniforme, secreta, de longitud suficiente y utilizada una vez.

- **Seguridad computacional:** un ataque puede existir en teoría, pero su ventaja debe ser despreciable para adversarios eficientes con recursos acotados.

2. Clasificación de criptosistemas modernos

Familia	Modelo de clave	Propiedad	Ejemplos
Cifrado simétrico	Secreto compartido	Confidencialidad; con AEAD también integridad	AES-GCM, ChaCha20-Poly1305
Establecimiento asimétrico	Par pública/privada	Transporte, acuerdo o encapsulación de material secreto pequeño	RSA-OAEP, ECDH/X25519, ML-KEM/HPKE
Firma digital	Privada firma; pública verifica	Autenticidad e integridad verificables públicamente	RSA-PSS, ECDSA, EdDSA
Hash	Sin clave	Huella fija y componente de protocolos	SHA-256, SHA-3
MAC	Secreto compartido	Integridad y autenticación simétrica	HMAC, Poly1305
KDF	Secreto o contraseña	Derivación, endurecimiento y separación de claves	HKDF, PBKDF2, Argon2id
KEM	Par pública/privada	Encapsulación de un secreto compartido	ML-KEM, componente de HPKE

También debe distinguirse entre **primitiva**, **esquema**, **protocolo** y **sistema**. AES es una primitiva; AES-GCM es una construcción AEAD; TLS 1.3 es un protocolo; una aplicación con identidad, almacenamiento y operación es un sistema.

Qué significa integridad

Un hash sin clave detecta cambios solo cuando su valor esperado llega por un canal confiable. Frente a un atacante activo, publicar **SHA-256(M)** junto con **M** no autentica nada: el atacante puede modificar ambos. Para autenticar se necesita una clave o una raíz de confianza, por ejemplo HMAC, una firma digital o un tag AEAD.

3. Criptografía simétrica

La misma clave secreta cifra y descifra. Es rápida y adecuada para datos voluminosos, pero exige resolver generación, distribución, almacenamiento, rotación, revocación y destrucción de claves.

AES

AES es un cifrador por bloques de **128 bits**. Sus claves pueden medir 128, 192 o 256 bits. El tamaño de bloque no cambia con la longitud de la clave.

ChaCha20

ChaCha20 es un cifrador de flujo con clave de 256 bits. Se utiliza normalmente junto con Poly1305 como construcción AEAD.

Gestión del secreto compartido

La eficiencia simétrica no elimina el problema de claves. Deben definirse:

- generación con un CSPRNG;
- distribución por un canal o protocolo autenticado;
- almacenamiento y controles de acceso;
- separación por propósito mediante KDF;

- rotación y período criptográfico;
- revocación y destrucción verificable;
- registro de identificadores de clave sin registrar el secreto.

4. Bits de seguridad

Para una clave ideal de k bits existen 2^k candidatos. Una búsqueda exhaustiva clásica necesita, en promedio, aproximadamente 2^{k-1} pruebas.

Los bits de seguridad representan el costo logarítmico del mejor ataque conocido. No tienen por qué coincidir con:

- longitud nominal de clave;
- longitud de una contraseña;
- tamaño de una salida hash;
- cantidad de caracteres;
- parámetros de un algoritmo.

Una contraseña humana larga puede tener poca entropía si sigue patrones. Derivar 256 bits desde ella no crea 256 bits de incertidumbre.

Impacto cuántico

Grover ofrece una aceleración cuadrática idealizada para búsqueda no estructurada. Shor amenaza factorización y logaritmo discreto, por lo que afecta RSA, DH y ECC.

Tabla orientativa

Nivel clásico aproximado	Lectura didáctica	Ejemplos y cautelas
56 bits	Insuficiente	DES puede recorrerse por fuerza bruta
80 bits	Heredado	No apropiado para protección nueva
112 bits	Margen clásico heredado	Debe evaluarse según vida útil y política de transición
128 bits	Objetivo moderno común	AES-128 y curvas correctamente elegidas se sitúan en este orden
192 bits	Margen alto	Mayor costo y tamaños; no corrige un protocolo defectuoso
256 bits	Margen muy alto clásico	AES-256 o clave ChaCha20; la seguridad del sistema puede ser menor

NIST SP 800-57 relaciona fortaleza, algoritmo, tamaño de clave y período de protección. No debe interpretarse una fila como garantía atemporal: el horizonte, la sensibilidad, la implementación y las transiciones normativas importan.

Ejemplo de cálculo

Si un atacante prueba r candidatos por segundo, recorrer un espacio ideal de 2^k tarda en promedio:

$$\text{tiempo medio} \approx 2^{(k-1)} / r$$

La fórmula solo describe fuerza bruta ideal. No modela sesgos humanos, ataques de canal lateral, reducción del espacio por estructura ni errores del protocolo.

5. Cifrado por bloques y por flujo

Bloques

Un cifrador por bloques transforma unidades fijas. Para mensajes largos necesita un modo de operación.

Flujo

Un cifrador de flujo genera un keystream que se combina con el mensaje mediante XOR.

Si se reutiliza el mismo flujo:

```
C1 = M1 XOR K
C2 = M2 XOR K
C1 XOR C2 = M1 XOR M2
```

La clave desaparece de la ecuación y se expone una relación entre mensajes.

6. Modos de operación

Modo	Garantía	Condición crítica
ECB	No oculta estructura repetida	Evitar
CBC	Confidencialidad si se usa correctamente	IV apropiado y MAC separado
CTR	Confidencialidad tipo flujo	Contador/nonce no repetido
GCM	AEAD: confidencialidad, integridad y AAD	Nonce único por clave
XTS	Protección de sectores	No sustituye AEAD general
ChaCha20-Poly1305	AEAD de flujo	Nonce único por clave y verificación del tag antes de usar el texto

La recomendación general para datos y protocolos es preferir AEAD, por ejemplo AES-GCM o ChaCha20-Poly1305.

Nonce, IV y contador

No son sinónimos universales. Un IV puede requerir imprevisibilidad, unicidad o ambas propiedades según el esquema. Un nonce significa “número usado una vez” dentro del alcance que define el protocolo. En GCM y ChaCha20-Poly1305, repetir el nonce con la misma clave puede destruir las garantías de confidencialidad y autenticidad.

Una estrategia segura debe especificar quién genera el nonce, cómo se evita la colisión, qué ocurre después de un reinicio, cómo se particiona el espacio entre emisores y cuándo se rota la clave.

Encrypt-then-MAC y AEAD

Una composición heredada segura puede cifrar y luego autenticar el ciphertext junto con metadatos relevantes usando claves separadas. AEAD integra esa interfaz y reduce el espacio para errores. AAD permite autenticar campos que deben permanecer visibles, como versión de protocolo, identificador de sesión o encabezado de paquete.

7. Padding

PKCS#7 completa el último bloque agregando n bytes cuyo valor es n . Si faltan cinco bytes, se añaden cinco bytes `05`.

El padding no autentica. Una aplicación que revela si el relleno es válido puede crear un **padding oracle**. Las defensas incluyen:

- preferir AEAD;

- verificar autenticidad antes de procesar;
- uniformar errores;
- evitar diferencias temporales observables.

Ejemplo PKCS#7

Para bloques de 16 bytes, una entrada de 11 bytes recibe cinco bytes `05`. Una entrada que ya mide exactamente 16 bytes recibe un bloque completo de dieciséis bytes `10`. Esto permite retirar el padding sin ambigüedad. CTR, GCM y ChaCha20 no necesitan este relleno porque operan como flujo sobre la longitud real.

8. Algoritmos simétricos

AES

- bloque fijo de 128 bits;
- claves de 128, 192 o 256 bits;
- adecuado con GCM para AEAD;
- XTS para almacenamiento por sectores;
- no usar ECB.

ChaCha20

ChaCha20 es un cifrador de flujo diseñado alrededor de tres operaciones sobre palabras de 32 bits:

- suma módulo 2^{32} ;
- XOR;
- rotación a izquierda.

Por eso se describe como una construcción **ARX**. No utiliza S-boxes ni tablas secretas. Su entrada IETF tiene:

```
4 palabras constantes | 8 palabras de clave
1 palabra contador | 3 palabras de nonce
```

Las 16 palabras forman un estado de 4 por 4. Cada quarter round actualiza cuatro palabras mediante sumas, XOR y rotaciones de 16, 12, 8 y 7 bits. Una ronda doble ejecuta primero cuatro quarter rounds por columnas y luego cuatro por diagonales. ChaCha20 aplica diez rondas dobles, es decir, 20 rondas.

Después de las rondas se suma el estado inicial palabra por palabra y se serializa en little-endian para producir un bloque de 64 bytes de keystream. El mensaje se cifra con XOR:

```
C = M XOR ChaCha20(K, contador, N)
M = C XOR ChaCha20(K, contador, N)
```

Con contador IETF de 32 bits no puede cifrarse indefinidamente bajo un único nonce. La implementación debe impedir el desbordamiento del contador. Para mensajes de varias partes, el protocolo debe asignar nonces sin repetición y límites explícitos.

Límite esencial: ChaCha20 solo aporta confidencialidad y es maleable. La construcción de uso normal es ChaCha20-Poly1305, que añade autenticación de ciphertext y AAD. No debe inventarse una composición manual.

Vector de control RFC 8439

El laboratorio usa la clave `000102...1f`, contador `1` y nonce `000000090000004a00000000`. El primer bloque esperado comienza con `10f1e7e4d13b5915` y termina con `b2503c4e`. La comparación exacta detecta errores de endianness, rotación, orden de rondas o suma final.

Reutilización de nonce en ChaCha20

Con la misma clave, nonce y contador se repite el keystream:

```
C1 XOR C2
= (M1 XOR KS) XOR (M2 XOR KS)
= M1 XOR M2
```

La igualdad no revela automáticamente ambos mensajes, pero elimina la máscara secreta y habilita crib-dragging, uso de formatos conocidos y recuperación progresiva. En ChaCha20-Poly1305, la reutilización también compromete la seguridad del autenticador.

ChaCha20-Poly1305

- ChaCha20 genera el flujo y una clave de un solo uso para Poly1305;
- Poly1305 autentica AAD, ciphertext y longitudes según un formato exacto;
- el receptor verifica el tag antes de liberar el texto claro;
- el nonce debe ser único por clave;
- XChaCha20-Poly1305 usa un nonce extendido en APIs que lo estandarizan, pero sigue exigiendo una biblioteca mantenida.

DES y 3DES/TDEA

DES debe tratarse como antecedente histórico: su clave efectiva de 56 bits es vulnerable a fuerza bruta. 3DES/TDEA también tiene bloques de 64 bits y límites de volumen que agravan colisiones. Según la transición de NIST SP 800-131A, TDEA no debe aplicarse para protección criptográfica nueva; su presencia debe limitarse a descifrado, recuperación o migración controlada de datos heredados.

Selección práctica

Contexto	Construcción preferible	Motivo
Protocolo general con aceleración AES	AES-GCM	AEAD estandarizado y eficiente
Software sin aceleración AES	ChaCha20-Poly1305	Buen rendimiento con operaciones ARX
Disco por sectores	AES-XTS	Diseñado para almacenamiento, sin autenticidad general
Interoperabilidad heredada	Migrar y encapsular	No crear datos nuevos con algoritmos retirados

9. Criptografía asimétrica

Usa un par de claves pública y privada. Su costo hace que normalmente se utilice para:

- acuerdo de claves;
- encapsulación;
- firmas;
- autenticación;
- protección de claves de sesión.

No se utiliza normalmente para cifrar archivos completos.

10. RSA y curvas elípticas

RSA

- **RSA-OAEP**: cifrado de material pequeño o transporte de una clave aleatoria al titular de la privada. No es un acuerdo interactivo de claves ni el KEM estandarizado de HPKE.
- **RSA-PSS**: firma.
- **Textbook RSA**: inseguro y determinista.

La seguridad depende del padding, el tamaño de módulo, la validación, la protección de la clave privada y la resistencia a canales laterales. "RSA" sin especificar el esquema no es una configuración completa.

ECC

- **ECDH/X25519**: acuerdo de claves.
- **ECDSA/EdDSA**: firma.
- **ECIES**: familia de composiciones híbridas basadas en acuerdo EC, KDF y cifrado simétrico.
- **HPKE**: KEM + KDF + AEAD.

La expresión "cifrado EC" debe interpretarse como una composición híbrida, no como una operación de curva que cifra directamente grandes volúmenes.

Comparación funcional

Mecanismo	Entrada pública	Resultado	Secreto hacia adelante
RSA-OAEP con pública estática	Clave pública del receptor	El emisor elige y transporta un secreto	No, si luego se compromete la privada histórica
ECDH/X25519 efímero autenticado	Públicas efímeras autenticadas	Ambas partes calculan el mismo secreto	Sí, si se borran los secretos efímeros
ML-KEM	Clave pública KEM	Encapsulador produce ciphertext y secreto; receptor desencapsula	Depende del protocolo y del uso de claves efímeras
HPKE	Suite KEM + KDF + AEAD	Contexto híbrido para cifrado a pública	Depende del modo y la gestión de claves

11. Claves de sesión

Una clave de sesión se limita a una conexión, archivo, dirección o intercambio. Que sea de corta duración reduce exposición y facilita rotación, pero no implica automáticamente secreto hacia adelante.

El patrón moderno es:

```
autenticación de claves públicas
-> transporte, acuerdo o KEM
-> KDF con contexto
-> claves separadas por dirección y propósito
-> AEAD con nonce y AAD
-> rotación y destrucción
```

Ejemplos:

```
ECDH -> HKDF -> AES-GCM
X25519 -> HKDF -> ChaCha20-Poly1305
ML-KEM -> KDF -> AEAD
RSA-OAEP -> clave aleatoria transportada -> AEAD
```

ECDH sin autenticación es vulnerable a man-in-the-middle. La identidad se establece mediante certificados, firmas, huellas verificadas, PSK u otro canal autenticado.

Separación y contexto

HKDF no “fortalece” una contraseña. Su función normal es extraer y expandir material de alta entropía. Los campos de contexto deben vincular protocolo, versión, roles, transcript y propósito. Deben derivarse claves diferentes para cifrar, autenticar, exportar o cada dirección del canal.

Secreto hacia adelante

Existe cuando comprometer una clave de identidad de largo plazo no permite recuperar sesiones pasadas. Requiere un establecimiento efímero autenticado y que los secretos efímeros ya no existan al momento del compromiso. Guardar el secreto efímero, reutilizarlo indefinidamente o transportar claves mediante RSA estático cambia esa conclusión.

12. Laboratorio: vulnerar contraseña

Una contraseña humana no es una clave uniforme. Un atacante offline puede probar candidatos sin consultar al servidor cuando posee:

- salt;
- parámetros;
- hash o clave derivada;
- información sobre hábitos humanos.

Estrategias

- diccionario;
- reglas y sustituciones;
- palabras + años;
- máscaras;
- PIN;
- fuerza bruta acotada.

Salt

La salt es pública, única y aleatoria. Evita precálculo compartido y hashes idénticos entre usuarios, pero no impide probar candidatos.

KDF de contraseña

PBKDF2, bcrypt, scrypt o Argon2id elevan el costo de cada intento. No crean entropía inexistente. Argon2id combina resistencia a ataques dependientes e independientes de datos y permite ajustar memoria, iteraciones y paralelismo.

Los perfiles de RFC 9106 sirven como punto de referencia, no como valores universales. La aplicación debe medir su plataforma, fijar un tiempo aceptable para autenticación legítima, reservar memoria compatible con concurrencia, almacenar los parámetros junto al verificador y poder actualizarlos. NIST SP 800-63B-4 recomienda un esquema resistente a ataques offline, salt y factor de costo apropiado.

Cadena de ataque

```
candidato
→ normalización y codificación
→ KDF con la misma salt y parámetros
→ comparación con objetivo
```

Recomendaciones

- Argon2id cuando sea posible;
- parámetros actualizables;
- salts únicas;
- gestor de contraseñas;
- MFA;
- límites y detección para ataques online;
- protección adicional de secretos de servidor cuando corresponda.

Laboratorio seguro

El laboratorio del módulo opera con contraseñas ficticias, espacio de búsqueda acotado y ejecución local. El objetivo es medir cómo cambian intentos y tiempo, no recuperar credenciales reales. Un ejercicio correcto debe:

1. generar una salt distinta por registro;
2. guardar algoritmo, parámetros, salt y verificador;
3. probar un diccionario didáctico limitado;
4. comparar candidatos con el mismo procedimiento;
5. documentar por qué el costo no reemplaza una contraseña fuerte;
6. terminar sin exportar datos ni conectarse a servicios externos.

13. Casos de diseño

Caso A: archivo local protegido por contraseña

```
contraseña UTF-8
-> KDF de contraseña con salt y parámetros
-> clave AES-GCM
-> nonce aleatorio único bajo esa clave
-> ciphertext + tag
-> paquete con versión, KDF, parámetros, salt, nonce y AAD
```

El paquete no necesita ocultar salt, nonce o parámetros. Sí necesita una codificación inequívoca, límites de tamaño, errores uniformes y autenticación de metadatos relevantes.

Caso B: canal cliente-servidor

```
validar identidad del servidor
-> X25519 efímero autenticado
-> transcript + secreto compartido
-> HKDF
-> claves por dirección
-> ChaCha20-Poly1305 o AES-GCM
-> números de secuencia y rotación
```

La autenticación evita MITM. Las claves efímeras y su eliminación permiten secreto hacia adelante. Los números de secuencia ayudan a construir nonces únicos y detectar reordenamiento según el protocolo.

Caso C: mensaje a una clave pública

HPKE combina KEM, KDF y AEAD. La aplicación todavía debe vincular la identidad correcta a la clave pública, elegir un modo apropiado, definir AAD y proteger contra reenvíos si ese riesgo pertenece al modelo.

14. Errores frecuentes

1. Decir “usa AES” sin especificar modo, autenticación, nonce y gestión de claves.
2. Tratar un hash público como prueba de autenticidad.
3. Reutilizar nonce en CTR, GCM o ChaCha20-Poly1305.
4. Liberar texto claro antes de verificar el tag.
5. Usar RSA de libro o cifrar datos voluminosos directamente con RSA.
6. Llamar “acuerdo” al transporte RSA-OAEP.
7. Suponer que toda clave de sesión da secreto hacia adelante.
8. Derivar muchas claves con la misma etiqueta de contexto.
9. Elegir parámetros de contraseña sin medir memoria, latencia y concurrencia.
10. Registrar claves, contraseñas, texto claro o errores demasiado informativos.

15. Actividades y autoevaluación

Actividad 1: clasificación

Para cada elemento - AES, AES-GCM, SHA-256, HMAC, RSA-OAEP, ECDH, ML-KEM, HKDF y TLS 1.3 - indique si es primitiva, esquema, construcción o protocolo; modelo de clave; propiedad; y un límite.

Actividad 2: bits de seguridad

Explique por qué una contraseña que se deriva a 256 bits no tiene necesariamente 256 bits de seguridad. Incluya distribución humana, tasa de prueba, salt y costo de KDF.

Actividad 3: nonce repetido

Use el laboratorio ChaCha20 con dos mensajes de igual longitud. Verifique $C1 \text{ XOR } C2 = M1 \text{ XOR } M2$ y explique qué información adicional permitiría recuperar texto.

Actividad 4: arquitectura híbrida

Diseñe un canal con autenticación, X25519 efímero, HKDF y AEAD. Identifique transcript, contexto, claves por dirección, nonces, rotación y condición de secreto hacia adelante.

Preguntas de control

1. ¿Qué diferencia hay entre confidencialidad e indistinguibilidad?
2. ¿Por qué un hash sin clave no autentica frente a un atacante activo?
3. ¿Qué propiedad agrega Poly1305 a ChaCha20?
4. ¿Qué rompe la reutilización de nonce?
5. ¿Cuándo se necesita padding y cuándo no?
6. ¿Por qué RSA-OAEP no es ECDH?
7. ¿Qué condiciones concretas exige el secreto hacia adelante?
8. ¿Por qué una salt pública sigue siendo útil?
9. ¿Qué parámetros deben calibrarse en Argon2id?
10. ¿Qué datos deberían formar parte de AAD?

16. Lista de comprobación curricular

Punto del programa	Cobertura principal	Recurso práctico
Fundamentos	Sección 1	Teoría HTML y mapas
Clasificación	Sección 2	Clasificador interactivo
Criptografía simétrica	Sección 3	Comparador simétrico
Bits de seguridad	Sección 4	Espacio de claves
Bloque y flujo	Sección 5	Bloques frente a flujo
Modos de operación	Sección 6	Modos AES y AEAD
Padding	Sección 7	Padding oracle controlado
Algoritmos simétricos	Sección 8	AES integral y ChaCha20
Criptografía asimétrica	Sección 9	Clasificador e híbrido
RSA y EC	Sección 10	RSA-OAEP y ECDH real
Claves de sesión	Sección 11	Cifrado híbrido
Vulnerar contraseñas	Sección 12	Ataque offline ficticio
ChaCha20	Sección 8	Vector RFC 8439 y nonce reuse

Recursos interactivos nuevos

1. Clasificación de criptosistemas modernos
2. AES, ChaCha20, modos y padding
3. ChaCha20 verificable
4. RSA, ECC, HPKE y claves de sesión
5. Ataque offline controlado a contraseñas ficticias
6. Mapas mentales del programa
7. Glosario integral
8. Cuestionario integral

Recursos interactivos existentes

1. Bloques frente a flujo
2. Contraseña, salt y KDF
3. Modos AES y AEAD
4. Cifrado local de archivos
5. Hash, HMAC y firma
6. Padding oracle
7. RSA, ECDH y cifrado híbrido

Bibliografía y estándares

1. NIST, **FIPS 197: Advanced Encryption Standard (AES)**, actualización editorial 2023.
<https://doi.org/10.6028/NIST.FIPS.197-upd1>

2. NIST, **SP 800-38A: Recommendation for Block Cipher Modes of Operation**.
<https://doi.org/10.6028/NIST.SP.800-38A>
3. NIST, **SP 800-38D: Recommendation for Galois/Counter Mode (GCM)**.
<https://doi.org/10.6028/NIST.SP.800-38D>
4. NIST, **SP 800-57 Part 1 Rev. 5: Recommendation for Key Management**.
<https://doi.org/10.6028/NIST.SP.800-57pt1r5>
5. NIST, **SP 800-131A Rev. 2: Transitioning the Use of Cryptographic Algorithms and Key Lengths**.
<https://doi.org/10.6028/NIST.SP.800-131Ar2>
6. NIST, **SP 800-63B-4: Authentication and Authenticator Management**.
<https://doi.org/10.6028/NIST.SP.800-63B-4>
7. Nir y Langley, **RFC 8439: ChaCha20 and Poly1305 for IETF Protocols**. <https://www.rfc-editor.org/rfc/rfc8439>
8. Moriarty et al., **RFC 8017: PKCS #1 v2.2**. <https://www.rfc-editor.org/rfc/rfc8017>
9. Krawczyk y Eronen, **RFC 5869: HKDF**. <https://www.rfc-editor.org/rfc/rfc5869>
10. Krawczyk y Wee, **RFC 9180: Hybrid Public Key Encryption**. <https://www.rfc-editor.org/rfc/rfc9180>
11. Biryukov et al., **RFC 9106: Argon2 Memory-Hard Function**. <https://www.rfc-editor.org/rfc/rfc9106>
12. NIST, **FIPS 202: SHA-3 Standard**. <https://doi.org/10.6028/NIST.FIPS.202>

Regla profesional: utilizar estándares vigentes, APIs de alto nivel, bibliotecas mantenidas, generación segura de claves, gestión de nonces y revisión de protocolo. Los laboratorios son educativos.

Material elaborado por el profesor Sergio Gevatschnaider.